Ok antigravity we are going to make a massive project so be precise, clear, and hardworking; no laziness, + be sharp. Ok, build an app called oceanstudio. Our course insight is that this application is built with the most powerful coding languages available for mobile development, a fully powerful, supported native language [Specify the native language, e.g., Kotlin for Android or Swift for iOS, or a cross-platform framework like Flutter or React Native]. This is a coding agent which works on a hardware level on a mobile device to code on hardware, so choose the right language which gives more advancements, capabilities, and features. Keep in mind that afterwards we might make this application copy a website format too, with supported features like that, but that would be later, in the end. For now, let's focus on core things. First the UI. See, connect the repo ocean.studio and setup your cloud agents first. Once everything is done, then let's go UI. in Firebase, and connect authentication. Build our authentication page with premium quality and design. White with soft and very little reddish or maybe yellowish tone, soft full white. Or white and black whatever, feels like a real modern site, no terminal-like fake UIs or mock-up simulations. The auth flow bar should be dynamically animated and transitionful, and must work. The user can sign up, sign in manually, can directly sign in with Google. Also, set up 'forget password' etc.; all must work already. Once granted, the user lands in the modern, clean UI landing page, with lot of text, footer, and detail about our features. And you can use background looped videos or images, or any 3D components from different services if you need, for an animated, eye-soothing design. You do not add useless cards like UI in the middle of the screen anywhere uselessly. You download stock videos or footages for quality and use them as needed for a premium site look.Then a 'launch workspace' button is given. Which first asks the user for a few configurations, like the UI of the templates of how it looks (3-4), and then what language like English or etc., any with good transition. Then a review-driven agent, auto agent, or bypassed agent. These features must work. The auto agent thinks before proceeding and is cautious on critical things that could harm. While the review-driven agent does everything on user's approval, and bypassed does all independently without any review or anything. I want this to work clean, just like you, Cursor, look how good you made it. I want my site to be better than Cursor, codex, antigravity ide, and have all of these functionalities and bit design.functionalities and more, with proper research—no simulation but actual workflow, just like big companies.First of all, then the main page opens. The user gets a side bar on the left, a clean thing, with not outline gradients but shadow, a grey bit of shadow on the outline. A scroll-driven scroll bar must work in the sidebar so features can be expanded and seen. Then in the middle, the user gets an editor or preview screen. The editor is where codes are present; the user can edit it, rewrite, delete, or make any changes to them by choosing the code file. Then it would show in that screen middle, cleanly phrased. And beside that at top in small px components, the user gets options like codebase and preview. The codebase shows all the codebase files etc., in the environment, cleanly real. And the preview shows a hardware-driven preview of sites, apps, websites, etc., on the mobile device. And the preview also has functionality to work on a port, if the user activates or whether our agent activates any port, via using its terminal. The user does not need to open the browser; it would directly see the port in our preview and use it whatever it is, smoothly. Full screen, give back environment to exit preview. In preview and codebase, it contains all code files, folders, dynamically, all with modern UI functionalities; like don't add emojis. Design everything: the user can expand code folders to see files or folders inside them

and expand other folders inside them to see more folders or files like full functionality. OK, this is done.
Then on the right side, add a right sidebar. This right sidebar plate is where our chatbox, agent chatbox, everything lives. In it, create down below at the bottom a curved, with no hard corners, chatbox, with a grey shadow outline, or maybe RGB rim lighting on the outline, in a loop, cleanly. A send button in the chatbox cleanly, and a '+' button to open some things I'll talk later. Then when the user sends any prompt or message, then the pause button shows to terminate the session, and the user can type again. I mean normal things; all AIs including you have these functionalities. Now the '+' icon. The '+' icon opens a list of things, like MCP connectors, webhooks, plugins, upload photo, upload file. This feature must work how other AIs work. However, we talk about plugin, webhooks, and MCP connectors later, but upload file and upload photo—these must work how other AIs work, including you, same logic. Do proper research, okay?
Now first, integrate a terminal in our app and agent internally. A fully native mobile terminal, like Termux or more than that, that runs full commands on the mobile's underlying operating system, and can run any commands from the internet; it has internet driven to do so. The user can install the binaries or compatibility layers they need. This is just a full terminal working on user's hardware. No simulation, no mock-up, all billion of binaries, everything, a fully configured terminal, where the user can run all terminal commands, install anything, add any binary or compatibility layers, just like a shell. Note: in the end, we might copyor maybe u can build by yourself with strtagy and more advanced comtpability and reach. This opens a separate terminal UI. The Native Terminal also connects to preview—I mean if user activates any port or any OS screen via the terminal, it would be activated in our preview, whatever port it is. You know terminal features, like clear all commands, use keyboard keys to interact with terminal, restart session, view port, etc. As mobile users don't have a hardware keyboard to do Ctrl or etc., give UI buttons for them, so if anywhere else in small buttons you provide cleanly, then it would be helpful. And also add an 'upload file' button to directly upload any user's file into the app's internal sandbox storage, so when the user opens any port, VNC, etc., anything, the file in the file storage of it would be available. It would be nice UI, grid, no stupid simulation, and use right fonts, styles, right open source materials if needed. And workflows, no emojis trash gradient or any pulsing dots shits; don't add these UIs. No pulsing glowing dots etc., no neon shits etc., use human aesthetic not AI aesthetic, just like how other modern IDEs look, appears UIs etc.
And note our agent, the agent in our APK/app, would have direct internal access connection to the codebase, preview page, and the native terminal; it can access them, run commands, do anything with full advanced compatibility layer and features. Ok, first this build, then I'll say next what to do. Also note, build detailed agent.md, skill.md instructions for the agent. In these you mention his capabilities, how and what it can do, and have access to, and how to see it, do it, and interact with it internally. Also the UI when agent thinks or does anything, look like screenshot shown. Just like you, it shows what tasks it is doing with clear UI, and right text phrasing. The agent can independently create these in the chat; it gets... or maybe you already create that agent... use I mean how you work? How your and other coding agents work? Add improved UI. First, shows text and the arrows you see > this is expandable arrows which shows internal things what it did ran, the summary and logs, cleanly with text. The text appears bit grey and black with shimmering effect; it shows how many seconds it thought, what task he did,

what's done and what's doing. Running terminal commands shows text like, it shows heading surface-level text in pure text English, and when user clicks on it, it expands and shows the summary, live logs, clean, and a lot more you do by yourself. Use your skill on design guidance or any MCP connections to create right things. No mockup, no cheat codes to literally complete the build quickly without clear configuration." here prompt for the terminal configuration # MASTER PROMPT: OCEANTERMINAL (NATIVE ANDROID TERMINAL ENGINE)

OBJECTIVE
Build a complete, non-simulated, native Android Terminal Application named "OceanTerminal" (`com.ocean.terminal`) in Kotlin, C++, and Android NDK. The application must operate as a real POSIX terminal environment using native pseudo-terminals (PTY), executing actual Linux system calls and binaries within an isolated user-space sandbox (`/data/data/com.ocean.terminal/files/usr`). Do NOT create mock text boxes or simulated command parsers.

---

1. DESIGN SPECIFICATION (WHITE STUDIO AESTHETIC)
- **Background**: Pure Paper White (`#FFFFFF`).
- **Default Text Color**: Deep Charcoal (`#1E293B`).
- **Prompt Identifier**: Crisp Ocean Blue (`#0284C7`).
- **Accent / Selection**: Soft Cyan (`#E0F2FE`) with High-Contrast Blue highlight (`#0369A1`).
- **Font**: Monospace JetBrains Mono or Fira Code (with fallback to Android `monospace`).
- **Soft Keyboard Bar**: Light translucent bar (`#F8FAFC`) with elevated action keys (`Ctrl`, `Alt`, `Tab`, `Esc`, `|`, `~`, `-`, `▲`, `▼`, `◄`, `►`).

---

2. NATIVE CORE & C++ PTY ARCHITECTURE (NDK)
1. **CMake & C++ PTY Driver (`native-pty.cpp`)**:
   - Implement `JNI` bindings to wrap `openpty()` and `forkpty()`.
   - Function `createSubprocess(String cmd, String cwd, String[] env)`:
     - Spawns a real sub-process under Android's POSIX kernel interface.
     - Passes standard file descriptors (`in`, `out`, `err`) to master PTY file descriptor.
     - Implements `setWindowSize(int fd, int rows, int cols, int widthPx, int heightPx)` using `ioctl(fd, TIOCSWINSZ, &ws)`.
   - Implement native file descriptor reader/writer thread loops to handle asynchronous terminal I/O streams.

2. **Environment & Sandbox Setup (`TerminalEnv.kt`)**:
   - Set internal storage root: `PREFIX = "/data/data/com.ocean.terminal/files/usr"`.
   - Set home directory: `HOME = "/data/data/com.ocean.terminal/files/home"`.
   - Configure Environment Variables passed to subprocess:
     ```
     ```

```
PREFIX=/data/data/com.ocean.terminal/files/usr
HOME=/data/data/com.ocean.terminal/files/home
PATH=$PREFIX/bin:$PREFIX/bin/applets
LD_LIBRARY_PATH=$PREFIX/lib
TERM=xterm-256color
COLORTERM=truecolor
LANG=en_US.UTF-8
```

---

3. BOOTSTRAP ENGINE & FIRST LAUNCH LOGIC
1. **Assets Package**: Include a compressed bootstrap archive (`bootstrap-arm64.zip`) inside `app/src/main/assets/` containing a lightweight BusyBox / Android POSIX utility set (`sh`, `ls`, `cd`, `mkdir`, `cat`, `ps`, `tar`, `gzip`, `curl`).
2. **First-Run Extraction (`BootstrapInstaller.kt`)**:
   - Verify if `$PREFIX/bin/sh` exists.
   - If missing:
     - Extract `bootstrap-arm64.zip` directly into `/data/data/com.ocean.terminal/files/usr`.
     - Recursively apply `chmod 700` (or Android File API `setExecutable(true)`) on all executables in `$PREFIX/bin`.
     - Initialize `$HOME` directory.

---

4. UI TERMINAL EMULATOR WIDGET (`TerminalActivity.kt`)
1. **Terminal Emulator View**:
   - Build/integrate a custom `TerminalView` extending `View` or `SurfaceView` that handles ANSI escape sequences (VT100, xterm-256color).
   - Render incoming stream bytes using a custom text buffer grid (`TerminalBuffer`).
   - Map user soft-keyboard key events (including `Ctrl` and `Alt` modifiers) into ASCII control codes (e.g., `Ctrl+C` -> `0x03`, `Ctrl+Z` -> `0x1A`).
2. **Input Session**:
   - Connect `TerminalView` directly to the `NativePTY` file descriptor input/output streams.
   - Support pinch-to-zoom font scaling and text selection/clipboard actions.

---

5. GRADLE & BUILD CONFIGURATION (`build.gradle.kts`)
- Target SDK: 34 (Android 14+).
- Enable NDK CMake build configuration:
  ```kotlin
  externalNativeBuild {
      cmake {
  ```

```
        path = file("src/main/cpp/CMakeLists.txt")
        version = "3.22.1"
    }
 }
 ndk {
    abiFilters.addAll(listOf("arm64-v8a", "x86_64"))
 }
```

Note we are making an best app, only apk, don't go else where else, and see the image for auth ui understanding i uploaded an image here auth token firebase // Import the functions you need from the SDKs you need

```
import { initializeApp } from "firebase/app";
import { getAnalytics } from "firebase/analytics";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
// For Firebase JS SDK v7.20.0 and later, measurementId is optional
const firebaseConfig = {
  apiKey: "AIzaSyAA1Kuhr6rFOHn-1KyTlgjY5bgzVR15YqY",
  authDomain: "oceanstudio-ef4c5.firebaseapp.com",
  projectId: "oceanstudio-ef4c5",
  storageBucket: "oceanstudio-ef4c5.firebasestorage.app",
  messagingSenderId: "2746278315",
  appId: "1:2746278315:web:e7b281fe41fcc5a875b2ee",
  measurementId: "G-DL1GQE56FL"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
const analytics = getAnalytics(app);
```